

Deployment Guide — Neuromorphic Sleep Stage Scoring

Deployment Overview

The final model (Improved Student, 99,477 parameters) is designed for edge deployment on resource-constrained devices. This guide covers export, optimization, and deployment strategies.

Current Deployment Status

Property	Value
Framework	PyTorch
Checkpoint	<code>artifacts/student_improved_best.pt</code>
Parameters	99,477
Model size (FP32)	~400 KB
CPU latency	8.5 ms/batch
Input shape	[1, 10, 4, 3000]
Output shape	[1, 10, 5]

LoRA Adapter Deployment

Adapter Properties

Property	Value
Adapter rank	8
Trainable parameters	552
Adapter size (FP32)	~2.2 KB
Base + adapter size	~402 KB
Latency overhead	~0 ms (negligible)

Deployment Options

Option 1: Unmerged (adapter loaded at runtime)

Base model (400 KB) + Adapter (2.2 KB) = 402 KB
Latency: 5.66 ms/batch

Option 2: Merged (adapter baked into base)

Merged model (400 KB)
Latency: 5.62 ms/batch

Both options produce identical predictions (verified: merge diff = 0.00e+00).

Adapter Reload Verification

```
from sleep_staging.adaptation import load_adapter

# Save adapter
save_adapter(model, "artifacts/lora/head_r8")

# Load into fresh base
model = ImprovedStudent(config)
model.load_state_dict(base_checkpoint)
model = apply_lora(model, lora_config)
load_adapter(model, "artifacts/lora/head_r8")
# Predictions match original (verified: diff = 0.00e+00)
```

Deployment Pipeline

PyTorch Checkpoint



- | |
|-------------------------------|
| 1. Model Loading & Validation |
| - Verify parameter count |
| - Check input/output shapes |
| - Test inference |



- | |
|-----------------------------|
| 2. Export to ONNX |
| - Fixed input shapes |
| - Operator version: 17 |
| - Dynamic batching disabled |



- | |
|------------------------|
| 3. ONNX Optimization |
| - Graph simplification |
| - Operator fusion |
| - Constant folding |



- | |
|----------------------------|
| 4. Quantization |
| - INT8 post-training |
| - Calibration dataset |
| - Per-channel quantization |



- | |
|------------------------|
| 5. Target Deployment |
| - Edge CPU / MCU |
| - Real-time inference |
| - On-device processing |

Step 1: Export to ONNX

Python Export Script

```
import torch
from src.models.improved_student import ImprovedStudent

# Load model
model = ImprovedStudent(n_classes=5)
checkpoint = torch.load("artifacts/student_improved_best.pt", weights_only=True)
if isinstance(checkpoint, dict) and "model_state_dict" in checkpoint:
    model.load_state_dict(checkpoint["model_state_dict"])
else:
    model.load_state_dict(checkpoint)
model.eval()

# Dummy input: [batch=1, seq_len=10, channels=4, samples=3000]
dummy = torch.randn(1, 10, 4, 3000)

# Export
torch.onnx.export(
    model,
    dummy,
    "artifacts/student_improved.onnx",
    opset_version=17,
    input_names=["psg_sequence"],
    output_names=["sleep_stages"],
    dynamic_axes=None, # Fixed shapes for edge deployment
)

print("Exported to ONNX: artifacts/student_improved.onnx")
```

Verify ONNX Export

```

import onnxruntime as ort
import numpy as np

# Load ONNX model
session = ort.InferenceSession("artifacts/student_improved.onnx")

# Test inference
dummy = np.random.randn(1, 10, 4, 3000).astype(np.float32)
outputs = session.run(None, {"psg_sequence": dummy})

print("ONNX output shape:", outputs[0].shape)
print("PyTorch vs ONNX match:", np.allclose(outputs[0], pytorch_output, atol=1e-5))

```

Step 2: Post-Training Quantization

INT8 Quantization

```

from onnxruntime.quantization import quantize_dynamic, QuantType

quantize_dynamic(
    model_input="artifacts/student_improved.onnx",
    model_output="artifacts/student_improved_int8.onnx",
    weight_type=QuantType.QInt8,
)

print("Quantized model saved to artifacts/student_improved_int8.onnx")

```

Quantization Impact

Model	Size	Accuracy	Latency
FP32	~400 KB	87.5%	8.5 ms
INT8	~100 KB	~86-87%	~3-4 ms

Step 3: Edge Deployment Targets

Supported Platforms

Platform	Tool	Notes
Linux/Mac/Windows	ONNX Runtime	General-purpose inference

Platform	Tool	Notes
ARM Cortex-M	TensorFlow Lite Micro	MCU deployment
Raspberry Pi	ONNX Runtime	Low-cost edge device
ESP32	TensorFlow Lite Micro	IoT deployment
NVIDIA Jetson	TensorRT	GPU-accelerated edge

Example: Raspberry Pi Deployment

```
# Install ONNX Runtime on Raspberry Pi
pip install onnxruntime

# Run inference
python scripts/infer.py \
    --checkpoint artifacts/student_improved_int8.onnx \
    --input demo/sample_inputs/sample_epoch.npz
```

Example: MCU Deployment (TensorFlow Lite Micro)

```
# Convert ONNX to TFLite
import onnx
from onnx_tf.backend import prepare
import tensorflow as tf

# ONNX to TF
onnx_model = onnx.load("artifacts/student_improved_int8.onnx")
tf_rep = prepare(onnx_model)
tf_rep.export_graph("model_tf")

# TF to TFLite
converter = tf.lite.TFLiteConverter.from_saved_model("model_tf")
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()

with open("model_int8.tflite", "wb") as f:
    f.write(tflite_model)
```

Step 4: Real-Time Inference

Input Pipeline

PSG Acquisition (100 Hz)



```
| Ring Buffer (300 seconds) |
| - Stores 10 × 30s epochs |
| - Overwrites oldest epoch |
```



```
| Preprocessing |
| - Bandpass: 0.5-35 Hz |
| - Notch: 50 Hz |
| - Z-score normalization |
```



```
| Model Inference |
| - Input: [1, 10, 4, 3000] |
| - Output: [1, 10, 5] |
| - Latency: ~8.5 ms |
```



```
| Output Processing |
| - argmax → class index |
| - Confidence thresholding |
| - Smoothing (optional) |
```



Sleep Stage Prediction
(Wake, N1, N2, N3, REM)

Latency Budget

Component	Time	Percentage
Preprocessing	~2 ms	23%
Model inference	~8.5 ms	77%
Total	~10.5 ms	100%

At 100 Hz sampling, the system can process epochs in real-time with significant headroom.

Hardware Recommendations

Minimum Requirements

Resource	Minimum	Recommended
RAM	2 MB	4 MB
Flash	500 KB	1 MB
CPU	ARM Cortex-M4	ARM Cortex-M7
Clock	100 MHz	200 MHz

Recommended Development Platforms

Platform	Cost	RAM	Notes
Raspberry Pi Zero 2 W	\$15	512 MB	Good for prototyping
Arduino Nano 33 BLE	\$30	256 KB	MCU-class device
STM32H7 Discovery	\$25	1 MB	High-performance MCU
Nordic nRF5340	\$10	512 KB	Low-power IoT

Deployment Checklist

- ☐ Model exported to ONNX
 - ☐ ONNX model verified against PyTorch output
 - ☐ INT8 quantization applied
 - ☐ Quantization accuracy validated
 - ☐ Target platform selected
 - ☐ ONNX Runtime / TFLite installed on target
 - ☐ Preprocessing pipeline implemented on target
 - ☐ Real-time inference tested
 - ☐ Latency measured on target hardware
 - ☐ Memory usage profiled
-

Future Work

- Quantization-Aware Training (QAT):** Train with quantization simulation for better INT8 accuracy
 - Pruning:** Remove redundant weights to further reduce model size
 - Knowledge Distillation to Smaller Models:** Distill into <50K parameter models
 - ONNX Runtime Mobile:** Optimize for mobile devices
 - WebAssembly:** Browser-based inference for web demos
-

